

Parciales viejos (MD)

Parcial 18/06/2025

Programación Concurrente ATIC/Redictado Programación Concurrente - Primera fecha de MD - 18/06/2025 - Tema 1

1. Resolver con PASAJE DE MENSAJES ASINCRÓNICOS (PMA) el siguiente problema. Hay *E estudiantes* que rinden un examen final y *un profesor* que cuando todos los estudiantes han llegado les entrega el enunciado. Luego el profesor va corrigiendo los exámenes y enviando la nota de acuerdo con orden en que se van entregando. Cada alumno realiza el examen, lo entrega y espera a que el profesor le indique la nota. *Nota:* maximizar la concurrencia; no realizar *busy waiting*; todos los procesos deben terminar.
2. Resolver con PASAJE DE MENSAJES SINCRÓNICOS (PMS) el siguiente problema. En un consultorio hay *un médico* para atender a *15 pacientes* de acuerdo con el orden de llegada. Cada paciente al llegar espera a que el médico lo atiende y le indique su tratamiento. *Nota:* existe la función *atender()* que simula que el médico está atendiendo a un paciente; todos los procesos deben terminar.
3. Resolver con ADA el siguiente problema. En una competencia de programadores hay *P participantes* y *UN coordinador*. El coordinador entrega el problema a resolver a los participantes y recibe las resoluciones para corregir. Cada participante debe conocer el resultado de su trabajo y el orden en que entregó su resolución. *Nota:* maximizar la concurrencia.

PMA

```
chan BARRERA(int);
chan ENVIAR_ENUNCIADO[N](str);
chan ENVIAR_EXAMEN(str, int);
chan ENVIAR_NOTA[N](int);

PROCESS Estudiante [idE: 1..E]
{
    examen exam;
    enunciado enun;
    int nota;

    // Barrera
    send BARRERA(1);
    receive ENVIAR_ENUNCIADO[idE](enun);
    // Hacer examen
    exam = resolverExamen(enun);
    send ENVIAR_EXAMEN(exam);
    receive EVIAR_NOTA[idE](nota);
}
```

PROCESS Profesor

```

{
    cola<str, int> cola_examenes;
    int i = 0, c = 0;
    int id, nota;
    examen exam;
    enunciado enun;

    // Barrera
    for i:= 1 to 50 do begin
        receive BARRERA();
    end;

    enun = generarEnunciado();

    // Para maximizar la concurrencia, podría estar tanto enviando enunciado
s,
    // como recibiendo exámenes y enviando notas
    do (i < 50) send ENVIAR_ENUNCIADO[i++](enun); // Enviar enunciado
    // Recibir exámenes
    [] (c < 50) receive ENVIAR_EXAMEN(id, exam) cola_examenes.push((id, e
xam));

        c++;

    // Enviar notas
    [] (not empty(cola_examenes)) id, exam = cola_examenes.pop();
        nota = corregirExamen(exam);
        send ENVIAR_NOTA[id](nota);

    od
}

```

[Revisar]

PMS

```

PROCESS Paciente [id: 1..15]
{
    tratamiento trat;

```

```

    // Cada paciente al llegar espera a que el médico lo atiende
    // y le indique su tratamiento
    Buffer!recibir_id(id);
    Medico?comenzar_atencion();
    // Recibir el tratamiento
    Medico?recibir(trat);
}

// Hay orden de llegada
PROCESS Buffer
{
    int id;
    int i = 0; int p = 0;
    cola<int> cola_ids;

    do (i < 15) Paciente[*]?recibir_id(id) → cola_ids.push(id); i++;
    [] (p < 15 && not empty(cola_ids)) Medico?pedido() →
                                                Medico!enviar_id(cola_id
s.pop();
                                                p++;

    od
}

PROCESS Medico
{
    int id;
    tratamiento trat;

    for i:= 1 to 15 do begin
        // Recibir el id de un paciente
        Buffer!pedido();
        Buffer?enviar_id(id);
        Paciente[id]!comenzar_atencion();
        // Atender al paciente
        trat = atender(id);
    end
}

```

```

        // Enviarle su tratamiento
        Paciente[id]!recibir(trat);
    end;
}

```

[Revisar]

ADA

Procedure EjADA is

Task Coordinador is

RecibirProblema(prob: OUT problema);

EntregarSolucion(sol: IN solucion; nota: OUT int; pos: OUT int);

End Coordinador;

Task Type Participante is

End Participante;

arrParticipantes: array (1..P) of Participante;

Task Body Participante is

prob: problema;

sol: solucion;

nota: int;

pos: int;

Begin

// Recibir problema

Coordinador.RecibirProblema(prob);

sol:= resolverProblema(prob);

// Entregar solución y obtener resultado

Coordinador.EntregarSolucion(sol, nota, pos);

End Participante;

Task Body Coordinador is

```

    posicion: int;
    probC: problema;
Begin
    posicion:= 0;
    probC:= generarProblema();

    for i:= 1 to 2*P loop
        SELECT
            ACCEPT RecibirProblema(prob: OUT problema) IS
                prob:= probC;
            END RecibirProblema;
        OR
            ACCEPT EntregarSolucion(sol: IN solucion; nota: OUT int: p
os: OUT int) IS
                nota:= corregirSolucion(sol);
                pos:= posicion;
            END EntregarSolucion;
            posicion:= posicion + 1;
        END SELECT;
    end loop;

    End Coordinador;
Begin
    null;
End;

```

[Revisar]

Parcial 4/11/2024

1) Las unidades postales (UP) son sucursales de correo que se ocupan habitualmente de recibir y entregar paquetes. En particular, la UP 16 sólo se ocupa de entregar paquetes a las personas que se presentan a retirarlos. Los clientes son atendidos por orden de llegada por el primer empleado que se encuentre libre. Al solicitar atención, el cliente le muestra un código QR al empleado que lo atiende, el cual le entrega el paquete como respuesta. Implemente un programa que permita modelar el funcionamiento de la UP 16 usando PMA, asumiendo que hay N clientes que retiran un único paquete y 3 empleados para atenderlos. Además, asuma que no habrá códigos QR erróneos ni paquetes faltantes. Notas: la función `obtenerQR(idCliente)` retorna el código QR asociado al identificador de cliente recibido como entrada. De forma similar, `obtenerPaquete(qr)` retorna el paquete asociado al QR recibido como entrada.

2) Resuelva el mismo problema anterior pero ahora usando PMS.

PMA

```
// PRE-CONDICIONES
```

```
// - No hay códigos QR erróneos ni paquetes faltantes
```

```
// - La función obtenerQR(idCiente) retorna el código QR asociado al identificador de cliente recibido como entrada
```

```
// - La función obtenerPaquete(qr) retorna el paquete asociado al QR recibido como entrada.
```

```
// - El cliente no hace otra cosa además de solicitar atención y recibir el paquete
```

```
// - El while (true) va a terminar cuando sea necesario
```

```
chan SOLICITUD(int, codigo_qr);
```

```
chan RESPUESTA[N](paquete);
```

```
PROCESS Cliente [idC: 1..N]
```

```
{
```

```
    codigo_qr codigo;
```

```
    paquete paq;
```

```
    // Generar código QR
```

```
    codigo = obtenerQR(idC);
```

```
    // "Al solicitar atención, el cliente le muestra un código QR al empleado que lo atiende"
```

```
    send SOLICITUD(idC, codigo);
```

```
    // Recibir paquete por medio de un canal privado
```

```

        receive RESPUESTA[idC](paq);
    }

PROCESS Empleado [1..3]
{
    codigo_qr codigo;
    idC int;
    paquete paq;

    while (true) {
        // Acá no pasa nada con que los tres empleados reciban elementos
        // desde el mismo canal, no hay conflicto porque →
        receive SOLICITUD(idC, codigo_qr);
        paq = obtenerPaquete(codigo_qr);
        send RESPUESTA[idC](paq);
    }
}

```

[\[Consultar\]](#)

PMS

```

// PRE-CONDICIONES
// - No hay códigos QR erróneos ni paquetes faltantes
// - La función obtenerQR(idCiente) retorna el código QR asociado al identifica
dor de cliente recibido como entrada
// - La función obtenerPaquete(qr) retorna el paquete asociado al QR recibido
como entrada.
// - El cliente no hace otra cosa además de solicitar atención y recibir el paqu
ete
// - El while (true) va a terminar cuando sea necesario

PROCESS Cliente [idC: 1..N]
{
    codigo_qr codigo;
    paquete paq;

```

```

// Generar código QR
codigo = obtenerQR(idC);

Coordinador!enviar(idC, codigo);

// Recibir paquete
Empleado?recibir(paq);
}

// Surje como intermediario entre los clientes y los empleados
PROCESS Coordinador
{
    codigo_qr codigo;
    int idC;
    cola<int, codigo> cola;

    // ¿Hace falta que esto se haga de manera no determinística?
    while (true) {
        if Cliente[*]?enviar(idC, codigo)
            push(cola, idC, codigo);
        [] not empty(cola) Empleado[*]?aviso_libre(idE)
            idC, codigo = pop(cola);
            Empleado[idE]?recibir_id_cod(idC, codigo);
    }
}

PROCESS Empleado [idE: 1..3]
{
    codigo_qr codigo;
    int idC;
    paquete paq;

    while (true) {
        Coordinador!aviso_libre(idE);
        Coordinador[id]?recibir_id_cod(idC, codigo);
        paq = obtenerPaquete(codigo);
    }
}

```



```

        Cliente[idC]!recibir(paq);
    }
}

```

[Corregido]

ADA

3) El problema *WordCount* consiste en calcular la cantidad de veces que aparece una palabra determinada en una porción, documento o archivo de texto. Implemente una solución a *WordCount* en ADA, considerando que se cuenta con una base de datos de texto distribuida en N partes. Además, existe un administrador que determina la palabra que se desea buscar y se la envía a las N tareas contadoras para que la busquen en la parte de la base de texto que poseen. Finalmente, el administrador calcula la cantidad total de apariciones. Notas: las tareas contadoras cuentan con la función *contarPalabras(pal)*, la cual retorna la cantidad de veces que aparece la palabra *pal* en su parte de la base de datos. El administrador cuenta con la función *obtenerPalabra()* que retorna la palabra a buscar. El problema se resuelve una única vez y su solución debe maximizar la concurrencia.

```

procedure WordCount is

```

```

    // Declaraciones

```

```

    Task Administrador is

```

```

        Entry ObtenerPalabra(palabra: OUT str);

```

```

        Entry EnviarCuenta(cuenta: IN int);

```

```

    End;

```

```

    Task Type Contador;

```

```

    // Implementaciones

```

```

    arrContadores: array (1..N) of Contador;

```

```

    Task Body Administrador is

```

```

        palabra: str;

```

```

        total: int;

```

```

    Begin

```

```

palabra:= obtenerPalabra();
total:= 0;

for i:= 1 to 2 * N do loop
    // Maximizar concurrencia permitiendo que se envíe
    // la palabra o que se reciba la cantidad que obtuvo
    // algún contador de manera no determinística
    SELECT
        ACCEPT ObtenerPalabra(palabra: OUT str) DO
            palabra:= palabra;
        END ObtenerPalabra;
    OR
        ACCEPT EnviarCuenta(cuenta: IN int) DO
            total:= total + cuenta;
        END EnviarCuenta;
    END SELECT;
end loop;

End Administrador;

Task Body Contador is
    cuenta: int;
    palabra: str;
    porcion_bd: db;
Begin
    // Invertir la comunicación para no tener que usar ids
    Administrador.ObtenerPalabra(palabra);

    cuenta:= contarPalabra(palabra);

    // Decirle al administrador la cantidad de veces que apareció
    // la palabra obtenida dentro de su porción de la base de
    // datos
    Administrador.EnviarCuenta(cuenta);

End Contador;

```

```
Begin
    null;
End WordCount;
```

[Corregido]

Parcial Segundo de 2024

PMA

1. La UNLP está organizando su maratón anual y P personas que deben pasar a retirar su remera y chip antes de la carrera. Para ello, los corredores deben dirigirse al edificio de Rectorado, donde habrá un organizador que los atenderá. Los corredores son atendidos de acuerdo con el orden de llegada, teniendo prioridad los corredores ancianos sobre los jóvenes y adultos.

Adicionalmente, los corredores con discapacidad tienen prioridad sobre todos los anteriores. Para ser atendidos, los corredores entregan su DNI al organizador, quien como respuesta les entrega la remera oficial y el chip asociado. Implemente un programa que permita resolver el problema anterior usando PMA. Notas: para conocer su prioridad, cada corredor puede llamar a la función `obtenerPrioridad()`, la cual retorna 0 si la persona tiene alguna discapacidad, 1 si es anciana, o 2 en otro caso; la función `obtenerRyC(dni)` retorna la remera y el chip asociado para el DNI recibido como entrada; la función `obtenerDNI()` retorna el DNI para el corredor que la invoca.

```
chan AVISAR(int);
chan ENTREGAR_DISC(int, int);
chan ENTREGAR_ANCI(int, int);
chan ENTREGAR_NORM(int, int);
chan RESPUESTA[N](remera, chip);
```

```
PROCESS Corredor [id: 1..P]
{
```

```

boolean es_discapacitado = ...; // Se asume que se sabe ya
int edad = ...; // Se asume que se sabe ya
int dni = obtenerDNI();
remera rem;
chip chi;

// Obtener prioridad
int prioridad = obtenerPrioriad(es_discapacitado, edad);

// Entregar chip al organizador
if (prioridad == 0)
    send ENTREGAR_DISC(dni, id);
else if (prioridad == 1)
    send ENTREGAR_ANCI(dni, id);
else
    send ENTREGAR_NORM(dni, id);

send AVISAR(1);

// Obtener la remera y el chip asociado
receive RESPUESTA[id](rem, chi);
}

```

PROCESS Organizador

```

{
    int ok;
    int dni;
    int id;
    remera rem;
    chip chi;

    for i:= 1 to P do
        receive AVISAR(ok);
        if (not empty(ENTREGAR_DISC))
            receive ENTREGAR_DISC(dni, id);
            rem, chi = obtenerRyC();

```

```

        send RESPUESTA[id](rem, chi);
    [] (empty(ENTREGAR_DISC) && not empty(ENTREGAR_ANCI))
        receive ENTREGAR_ANCI(dni, id);
        rem, chi = obtenerRyC();
        send RESPUESTA[id](rem, chi);
    [] (empty(ENTREGAR_DISC) && empty(ENTREGAR_ANCI) && not empty(ENTREGAR_NORM))
        receive ENTREGAR_NORM(dni, id);
        rem, chi = obtenerRyC();
        send RESPUESTA[id](rem, chi);
end;
}

```

[Corregido]

2. Las unidades postales (UP) son sucursales de correo que se ocupan habitualmente de recibir y entregar paquetes. En particular, la UP 16 sólo se ocupa de entregar paquetes a las personas que se presentan a retirarlos y cuenta con un único empleado. Como los clientes pueden tener que esperar para su atención, muchos aprovechan la plaza que está frente a la UP 16 para hacer ejercicio. Los clientes son atendidos por orden de llegada y se dividen en dos clases: los clientes "pacientes" que esperan a lo sumo 10 minutos a que el empleado los atienda y, pasado ese plazo, dan una vuelta a la plaza; y los clientes "ansiosos" que exigen atención inmediata y sino se van a dar una vuelta a la plaza. Más allá del tipo de cliente, luego de 3 intentos sin lograr que lo atiendan, el cliente se retira sin el paquete. El proceso de atención consiste en que el empleado le entrega un paquete al cliente, dado el código QR provisto por éste.

Implemente un programa que permita modelar el funcionamiento de la UP 16 usando ADA, asumiendo que hay N_1 clientes "pacientes" y N_2 clientes "ansiosos", y que todos retiran un único paquete. Además, asuma que no habrá códigos QRs erróneos ni paquetes faltantes. Notas: la función `obtenerQR()` retorna el

código QR asociado al cliente que la invoca; obtenerPaquete(qr) retorna el paquete asociado al QR recibido como entrada; la función caminata() modela la vuelta a la plaza.

```
procedure UP16 is
```

```
    Task Empleado is
```

```
        Entry AtenderPaciente(qr: IN codigo_qr; paq: OUT paquete);
```

```
        Entry AtenderAnsioso(qr: IN codigo_qr; paq: OUT paquete);
```

```
    End;
```

```
    Task Type ClientePaciente is
```

```
    End;
```

```
    Task Type ClienteAnsioso is
```

```
    End;
```

```
    arrPacientes: arr (1..N1) of ClientePaciente;
```

```
    arrAnsiosos: arr (1..N2) of ClienteAnsioso;
```

```
    Task Body Empleado is
```

```
    Begin
```

```
        for i:= 1 to (N1 + N2) loop
```

```
            SELECT
```

```
                ACCEPT AtenderPaciente(qr: IN codigo_qr; paq: OUT paqu
```

```
ete) DO
```

```
                    paq:= obtenerPaquet(qr);
```

```
                END AtenderPaciente;
```

```
            OR
```

```
                ACCEPT AtenderAnsioso(qr: IN codigo_qr; paq: OUT paqu
```

```
ete) DO
```

```
                    paq:= obtenerPaquet(qr);
```

```
                END AtenderAnsioso;
```

```
            END SELECT;
```

```
        end loop;
```

```
    End Empleado;
```

Task Body ClientePaciente is

```
    listo: boolean;  
    qr: codigo_qr;  
    paq: paquete;  
    veces: int;  
    id: int;
```

Begin

```
    listo:= false;  
    veces:= 3;  
    qr:= obtenerQR();
```

```
    while (veces > 0 && not listo) do loop
```

```
        SELECT
```

```
            Empleado.AtenderPaciente(qr, paq);
```

```
            listo:= true;
```

```
        DELAY 600
```

```
            caminata();
```

```
            veces:= veces - 1;
```

```
        END SELECT;
```

```
    end loop;
```

End;

Task Body ClienteAnsioso is

```
    listo: boolean;  
    qr: codigo_qr;  
    paq: paquete;  
    veces: int;  
    id: int;
```

Begin

```
    listo:= false;  
    veces:= 3;  
    qr:= obtenerQR();
```

```
    while (veces > 0 && not listo) do loop
```

```
        SELECT
```

```

        Empleado.AtenderPaciente(qr, paq);
        listo:= true;
    ELSE
        caminata();
        veces:= veces - 1;
    END SELECT;
end loop;
End;

Begin
    null
End UP16;

```

[Corregido]

Parcial Tercero de 2024

1. Existe un médico que debe auditar los estudios médicos de 15 pacientes que tienen un turno asignado con él. Los pacientes son atendidos de a uno por vez (nunca hay dos pacientes al mismo tiempo en el consultorio) y el orden de atención lo define su ID (1..15). El proceso de atención consiste en que el paciente entrega el estudio y el médico le responde con el resultado luego de revisarlo. Implemente un programa que permita resolver el problema anterior usando PMS. Notas: la función obtenerEstudio() retorna el estudio a revisar para el paciente que la invoca; la función revisarEstudio(estudio) retorna el resultado de revisar el estudio recibido como entrada.

```

PROCESS Paciente [1..15]
{}

PROCESS Buffer
{}

```


PROCESS Medico

```
{}
```

2. La UNLP está organizando su maratón anual y P personas que deben pasar a retirar su remera y chip antes de la carrera. Para ello, los corredores deben dirigirse al edificio de Rectorado, donde habrá un organizador que los atenderá. Los corredores son atendidos de acuerdo con el orden de llegada, teniendo prioridad los corredores ancianos sobre los jóvenes y adultos. Adicionalmente, los corredores con discapacidad tienen prioridad sobre todos los anteriores. Para ser atendidos, los corredores entregan su DNI al organizador, quien como respuesta les entrega la remera oficial y el chip asociado. Implemente un programa que permita resolver el problema anterior usando ADA. Notas: para conocer su prioridad, cada corredor puede llamar a la función `obtenerPrioridad()`, la cual retorna 0 si la persona tiene alguna discapacidad, 1 si es anciana, o 2 en otro caso; las funciones `obtenerRemera(dni)` y `obtenerChip(dni)` retornan la remera y el chip asociado para el DNI recibido como entrada, respectivamente; la función `obtenerDNI()` retorna el DNI para el corredor que la invoca; todas las tareas deben terminar.

Parcial 13/11/23

1. En una oficina existen 100 empleados que envían documentos para imprimir en 5 impresoras compartidas. Los pedidos de impresión son procesados por orden de llegada y se asignan a la primera impresora que se encuentre libre. Implemente un programa que permita resolver el problema anterior usando PASAJE DE MENSAJES ASINCRÓNICO (PMA).
- 2) Resuelva el mismo problema anterior pero ahora usando PASAJE DE MENSAJES SINCRÓNICO (PMS).
- 3) Resuelva con ADA el siguiente problema: la página web del Banco Central exhibe las diferentes cotizaciones del dólar oficial de 20 bancos del país, tanto para la compra como para la venta. Existe una tarea programada que se ocupa de actualizar la página en forma periódica y para ello consulta la cotización de cada uno de los 20 bancos. Cada banco dispone de una API, cuya única función es procesar las solicitudes de aplicaciones externas. La tarea programada consulta de a una API por vez, esperando a lo sumo 5 segundos por su respuesta. Si pasado ese tiempo no respondió, entonces se mostrará vacía la información de ese banco.

PMA

```
// Comunicación N a N con orden de llegada → Coordinador
// Cuando un proceso hace 2 tareas a la vez (evitar busy waiting) → Coordinador
```

```
chan ENVIO_DOCUMENTO(doc); // Empleado → Impresora
```

```
PROCESS Empleado [1..100]
{
    documento doc = cargarDocumento();
    send ENVIO_DOCUMENTO(doc);
}
```

```
PROCESS Impresora [1..5]
{
    documento doc;

    while (true) {
        receive ENVIO_DOCUMENTO(doc);
        imprimirDocumento(doc);
    }
}
```

- Si hay mas de 1 proceso B intentando agarrar recursos de un canal donde envian varios procesos A,

[Revisar]

PMS

```
PROCESS Empleado [1..100]
{
    documento doc = generarDocumento();
    Buffer!enviar(doc);
}
```

```
PROCESS Buffer
```

```

{
    cola<doc> cola;
    documento doc;
    int idImp;
    int pasaron = 100;

    do (pasaron > 0) Buffer?enviar(doc) →
        cola.push(doc);
    [] (pasaron > 0 && cola.size() > 0) Impresora[*]?aviso(idImp) →
        Impresora[idImp]!enviar_doc(d
oc);
        pasaron--;

    od

    // Por si tienen que terminar todos los procesos
    for i:= 1 to 5 do
        Impresora[i]!enviar_doc(null);
    }

PROCESS Impresora [idl: 1..5]
{
    documento doc = "";

    while (doc <> null) {
        Buffer!aviso(idl);
        Buffer?enviar_doc(doc);
        if (doc <> null)
            imprimirDocumento(doc);
    }
}

```

- Consultar si hace falta comunicarse de nuevo con el empleado
- Consultar si el proceso debe terminar

[Revisar]

ADA

Procedure EjADA is

Task Actualizador is
End Actualizador;

Task Type Banco is
 Entry ObtenerCotizacion(cot: OUT cotizacion);
End Banco;

arrBancos: array (1..20) of Banco;

Task Body Actualizador is
 cot: cotizacion;
Begin
 for i:= 1 to 20 loop
 SELECT
 arrBancos[i].ObtenerCotizacion(cot);
 OR DELAY 5
 cot:= "Vacío";
 END SELECT;

 // Asumir que existe una función que actualiza la página
 actualizarPagina(i, cot);
 end loop;
End Actualizador;

Task Body Banco is
Begin
 ACCEPT ObtenerCotizacion(cot: OUT cotizacion) IS
 cot:= obtenerCotizacion();
 END ObtenerCotizacion;
End Banco;

Begin

```
    null;  
End;
```

[Revisar]

Parcial 8/11/22

1. Resolver con PMA el siguiente problema. Se debe modelar el funcionamiento de una casa de venta de repuestos automotores, en la que trabajan V vendedores y que debe atender a C clientes. El modelado debe considerar que:
(a) cada cliente realiza un pedido y luego espera a que se lo entreguen; y (b) los pedidos que hacen los clientes son tomados por cualquiera de los vendedores. Cuando no hay pedidos para atender, los vendedores aprovechan para controlar el stock de los repuestos (tardan entre 2 y 4 minutos para hacer esto). **Nota:** maximizar la concurrencia.
2. Resolver con ADA el siguiente problema. Se quiere modelar el funcionamiento de un banco, al cual llegan clientes que deben realizar un pago y llevarse su comprobante. Los clientes se dividen entre los regulares y los premium, habiendo R clientes regulares y P clientes premium. Existe un único empleado en el banco, el cual atiende de acuerdo al orden de llegada, pero dando prioridad a los premium sobre los regulares. Si a los 30 minutos de llegar un cliente regular no fue atendido, entonces se retira sin realizar el pago. Los clientes premium siempre esperan hasta ser atendidos.

PMA

```
chan ENVIAR_PEDIDO(pedido, int); // Cliente → Coordinador  
chan AVISO(int); // Vendedor → Coordinador  
chan SIGUIENTE[V](pedido, int); // Coordinador → Vendedor  
chan RECIBIR_REPUESTO[C](repuesto); // Vendedor → Cliente
```

```
// Consultar → ¿Un solo pedido o infinitos?
```

```
PROCESS Cliente [idC: 1..C]
```

```
{  
    pedido ped = generarPedido();  
    send ENVIAR_PEDIDO(ped, idC);  
    repuesto rep;  
    receive RECIBIR_REPUESTO(rep);  
}
```

```
PROCESS Coordinador
```

```
{  
    pedido ped;  
    int idC;  
    int idV;  
  
    while (true) {
```

```

        // Despertarse al recibir un aviso de los vendedores
        receive AVISO(idV);
        if not empty(ENVIAR_PEDIDO) {
            receive ENVIAR_PEDIDO(ped, idC)
        } else {
            idC = -1;
            ped = null;
        }
        send SIGUIENTE[idV](ped, idC);
    }
}

PROCESS Vendedor [idV: 1..V]
{
    pedido ped;
    int idC;

    while (true) {
        send AVISO(idV);
        receive SIGUIENTE(ped, idC);
        // Tomar un pedido si hay
        if (ped <> null) repuesto rep = buscarRepuesto(ped);
            send RECIBIR_REPUESTO[idC](rep);
        // Si no hay pedido, controlar stock repuestos
        else
            // Tardan entre 2 y 4 minutos para hacer esto
            int tiempo = Math.random(120, 240);
            DELAY tiempo;
    }
}

```

- Comunicación N a N en PMA, y la parte que hace el procesamiento puede hacer una cosa u otra. En este caso hay que usar coordinador. Lo que pasa en este caso es que no interesa si llega un pedido del cliente o no, porque si no llega un pedido del cliente, el coordinador se lo va a hacer saber al empleado,

y este va a aprovechar para dormirse un tiempo, y si llega un pedido este se lo va a enviar el coordinador, y luego lo va a procesar.

[Corregido]

ADA

Procedure EjADA is

Task Type ClientePremium is
End ClientePremium;

Task Type ClienteRegular is
End ClienteRegular;

Task Empleado is
 Entry PedidoPremium(dinero: IN int; comprob: OUT comprobante);
 Entry PedidoRegular(dinero: IN int; comprob: OUT comprobante);
End Empleado;

arrPremium array (1..P) of ClientePremium;
arrRegulares: array (1..R) of ClienteRegular;

Task Body Empleado is
Begin
 loop
 SELECT
 ACCEPT PedidoPremium(dinero: IN int; comprob: OUT com
probante) IS
 comprob:= generarComprobante(dinero);
 END PedidoPremium;
 OR
 WHEN (PedidoPremium'count == 0) ⇒ ACCEPT PedidoReg
ular(dinero: IN int; comprob: OUT comprobante) IS
 comprob:= generarComprobante(dinero);
 END PedidoRegular;

```

        END SELECT;
    end loop;
End Empleado;

Task Body ClienteRegular is
    dinero: int;
    comprob: comprobante;
Begin
    dinero:= ...;
    // SELECT Temporal
    SELECT
        Empleado.PedidoRegular(dinero, comprob);
    OR DELAY 1800
        null;
    END SELECT;
End ClienteRegular;

Task Body ClientePremium is
    dinero: int;
    comprob: comprobante;
Begin
    dinero:= ...;
    Empleado.PedidoPremium(dinero, comprob);
End ClientePremium;

Begin
    null;
End;
```

[Corregido]